Open Source Audit Report

Chosen Software: Git

Submitted by:

Harsh Kumar Singh

Registration Number: 24BAI10057

Course: Open Source Software

Institution: VIT Bhopal

Date: March 31, 2026

1.Introduction

Open-source software plays a critical role in today's computing world. Most of the tools and technologies that developers use daily are built on open-source foundations. Unlike proprietary software, open-source systems allow users to study, modify, and distribute code, which promotes collaboration and transparency.

This project focuses on analyzing one such open-source tool, Git. Git is widely used in software development for tracking changes and managing code across teams. However, this report is not limited to technical features. It explores Git from multiple perspectives, including its origin, licensing model, ethical considerations, Linux system behavior, and its role in the broader open-source ecosystem.

Additionally, this project includes practical shell scripting tasks to demonstrate interaction with a Linux system and reinforce understanding of open-source environments.

2. Chosen Software: Git

Git is a distributed version control system used to track changes in source code during software development. It allows multiple developers to work on the same project efficiently by maintaining a complete history of changes.

Git is licensed under the GNU General Public License version 2 (GPL v2), which ensures that the software remains free and open for modification and redistribution.

I chose Git because it is deeply connected to Linux and open-source development. It is also widely used in both industry and academia, making it highly relevant. Its origin story and licensing model provide strong insights into the philosophy of open source.

3. Part A — Origin and Philosophy

3.1 A1 — The Problem Git Was Created to Solve

Before Git was introduced, managing large software projects was difficult, especially when multiple developers were involved. Traditional version control systems were mostly centralized, meaning there was a single server that stored all the code. This approach had limitations in terms of speed, reliability, and flexibility.

The Linux kernel, which is one of the largest open-source projects, initially used a proprietary tool called BitKeeper for version control. Although BitKeeper was efficient, it was not open source. This created a conflict because an open-source project was dependent on a closed-source tool.

Eventually, due to licensing disagreements, the Linux developers lost access to BitKeeper. This created an urgent need for a new system that was both powerful and open source.

Linus Torvalds, the creator of Linux, decided to build a new version control system from scratch.

Git was designed with specific goals in mind: high performance, strong data integrity, and support for distributed workflows. Unlike centralized systems, Git allows every developer to have a complete copy of the repository, including its history. This makes collaboration more flexible and reduces dependency on a central server.

Over time, Git became the standard for version control. It enabled platforms like GitHub and GitLab, which further simplified collaboration. Today, Git is not just a tool but a foundation for modern software development practices.

3.2 A2 — The License: What It Actually Says

Git is distributed under the GNU General Public License version 2 (GPL v2). This license is designed to ensure that software remains free in terms of user rights, not just cost.

The GPL license is based on four essential freedoms:

1. The freedom to run the program for any purpose
2. The freedom to study how the program works
3. The freedom to modify the program
4. The freedom to distribute copies

Git's license supports all these freedoms. However, it also introduces an important condition: if someone modifies the software and distributes it, they must also provide the source code under the same license.

This means that a company can use Git, modify it, and even sell it. However, they cannot take Git, modify it, and distribute it as closed-source software. Any distributed modifications must remain open.

This is where GPL differs from more permissive licenses like MIT. The MIT license allows developers to use the code in almost any way, including incorporating it into proprietary software without releasing the source code. In contrast, GPL ensures that the openness of the software is preserved in future versions.

In simple terms, MIT prioritizes flexibility, while GPL prioritizes long-term openness.

Another important concept is "free as in freedom" versus "free as in free beer." Git is not just free in terms of cost; it is free because users have control over how they use and modify it. This makes Git a strong example of software freedom in practice.

3.3 A3 — The Ethics of Open Source

Open source is not just a technical approach; it also raises important ethical questions.

One common question is whether all software should be open source. On one hand, open source promotes transparency, learning, and collaboration. It allows developers to understand how systems work and build on existing solutions. This can accelerate innovation and reduce duplication of effort.

On the other hand, not all software can realistically be open source. Companies invest significant resources in developing software and may rely on proprietary models to sustain their business. In such cases, making everything open source may not always be practical.

Another ethical issue is whether companies should be allowed to profit from open-source software. In my opinion, this is acceptable as long as companies follow the license terms and contribute back to the community in some way. Problems arise when companies take advantage of open-source projects without supporting them.

Regarding innovation, open source generally helps rather than harms it. When developers can access and modify existing code, they can focus on solving new problems instead of rebuilding basic tools. However, sustainability and funding remain challenges in many open-source projects.

Overall, I believe that while not all software needs to be open source, tools like Git benefit greatly from being open. Their transparency and flexibility make them more reliable and widely usable.

4. Part B — Linux Footprint

Git was installed on an Ubuntu-based Linux system using the apt package manager. The installation process was straightforward and required only a few commands.

The Git executable is located in /usr/bin/git, which is a standard directory for system binaries. Using the dpkg -L git command, it is possible to view all files associated with the Git package, including documentation and configuration files.

Git uses configuration files at different levels. The user-specific configuration is stored in ~/.gitconfig, while system-wide configurations may exist in /etc/gitconfig. This separation allows both global and user-specific settings.

Unlike services such as Apache or MySQL, Git does not run as a background process. It operates as a command-line tool and runs under the current user’s permissions.

Updates for Git are managed through the package manager. Using commands like apt policy git, users can check installed and available versions.

- Figure 1: Git installation and version check

```
harsh@HP: ~/oss-audit-24bai
harsh@HP:~/oss-audit-24bai10057$ pwd
/home/harsh/oss-audit-24bai10057
harsh@HP:~/oss-audit-24bai10057$ ls
README.md  notes  report  screenshots  scripts
harsh@HP:~/oss-audit-24bai10057$ git init
Reinitialized existing Git repository in /home/harsh/oss-audit-24bai10057/.git/
harsh@HP:~/oss-audit-24bai10057$ git remote -v
origin  https://github.com/harshkumarsingh200205-coder/oss-audit-24bai10057.git (fetch)
origin  https://github.com/harshkumarsingh200205-coder/oss-audit-24bai10057.git (push)
harsh@HP:~/oss-audit-24bai10057$ sudo apt update
sudo apt install git -y
[sudo] password for harsh:
Hit:1 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:2 http://archive.ubuntu.com/ubuntu jammy InRelease
Hit:3 http://archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:4 http://archive.ubuntu.com/ubuntu jammy-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
3 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
git is already the newest version (1:2.34.1-1ubuntu1.17).
0 upgraded, 0 newly installed, 0 to remove and 3 not upgraded.
harsh@HP:~/oss-audit-24bai10057$ git --version
git version 2.34.1
harsh@HP:~/oss-audit-24bai10057$
```

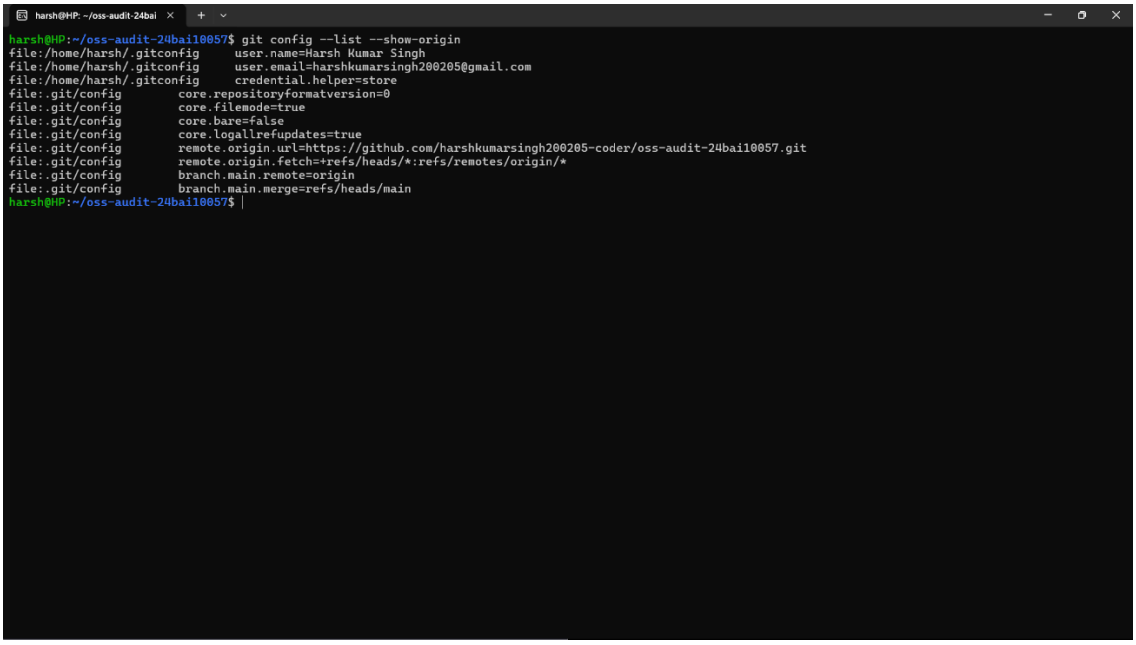
This screenshot shows that Git is successfully installed on the system. The version output confirms that the package is correctly configured and available through the Linux environment.

- Figure 2: Git executable path and package file listing

```
harsh@HP: ~/oss-audit-24bai
harsh@HP:~/oss-audit-24bai10057$ which git
/usr/bin/git
harsh@HP:~/oss-audit-24bai10057$ dpkg -L git | head -n 30
/.
/etc
/etc/bash_completion.d
/etc/bash_completion.d/git-prompt
/usr
/usr/bin
/usr/bin/git
/usr/bin/git-shell
/usr/lib
/usr/lib/git-core
/usr/lib/git-core/git
/usr/lib/git-core/git-add--interactive
/usr/lib/git-core/git-bisect
/usr/lib/git-core/git-daemon
/usr/lib/git-core/git-difftool--helper
/usr/lib/git-core/git-filter-branch
/usr/lib/git-core/git-http-backend
/usr/lib/git-core/git-http-fetch
/usr/lib/git-core/git-http-push
/usr/lib/git-core/git-imap-send
/usr/lib/git-core/git-instaweb
/usr/lib/git-core/git-merge-octopus
/usr/lib/git-core/git-merge-one-file
/usr/lib/git-core/git-merge-resolve
/usr/lib/git-core/git-mergetool
/usr/lib/git-core/git-mergetool--lib
/usr/lib/git-core/git-quiltimport
/usr/lib/git-core/git-remote-http
/usr/lib/git-core/git-request-pull
/usr/lib/git-core/git-sh-i18n
harsh@HP:~/oss-audit-24bai10057$
```

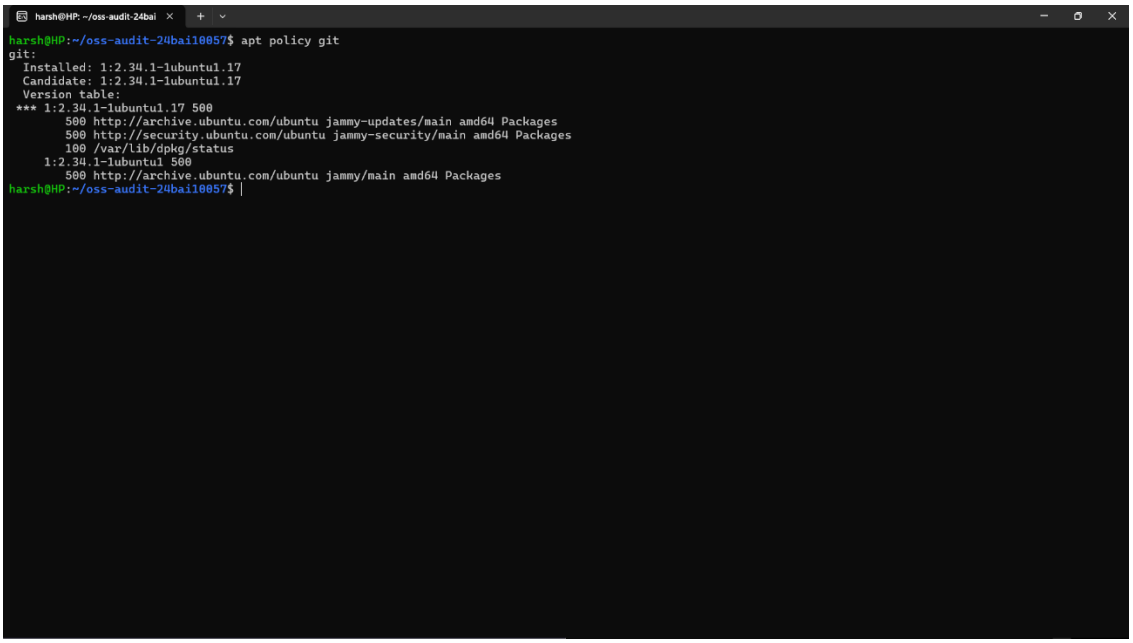
This output shows the location of the Git executable and the files installed by the package. It helps in understanding how Git is structured within the Linux filesystem.

- Figure 3: Git configuration source output



This screenshot shows the configuration files used by Git. It includes both user-level and system-level configuration, demonstrating how Git manages settings in a layered manner.

- Figure 4: Git package policy and update information



This output shows the installed and available versions of Git through the package manager. It demonstrates how updates are handled using Ubuntu repositories.

Service Management

Git does not run as a background service like Apache or MySQL. It is mainly used as a command-line tool and runs only when the user executes Git commands. Because of this, service management tools such as systemctl are generally not used for Git. This highlights that Git operates at the user level rather than as a continuously running system service.

Logs

Unlike server-based software, Git does not maintain a centralized log directory such as /var/log. Most of its activity is visible directly through terminal outputs or stored within repository metadata. This behavior reflects the fact that Git is not a daemon but a user-driven tool.

5. Part C — The FOSS Ecosystem

Git is not an isolated tool; it is part of a larger ecosystem of open-source technologies.

It works closely with tools such as SSH for secure communication and platforms like GitHub and GitLab for hosting repositories. These platforms have transformed how developers collaborate by introducing features like pull requests and issue tracking.

Git has enabled distributed development, where contributors from different locations can work on the same project independently. This has significantly improved the scalability of open-source projects.

Although Git is not a direct component of the LAMP stack, it plays an important role in developing and managing applications that run on such stacks. It is widely used in web development, system programming, and even machine learning projects.

The Git community is also highly active, with contributions from developers worldwide. This reflects the strength of open-source governance and collaboration.

6. Part D — Open Source vs Proprietary

Dimension	**Git**	**Perforce Helix Core**
License Model	GPL v2 (Open Source)	Proprietary
Cost	Free	Paid / Licensed
Source Code Access	Available to all users	Not publicly available
Security (Auditability)	Can be inspected and verified by anyone	Controlled internally by vendor
Support & Reliability	Community support + extensive documentation	Official enterprise support from vendor
Freedom to Modify	High (must follow GPL terms)	Restricted
Collaboration Model	Distributed version control	Mostly centralized enterprise workflows
Community vs Control	Community-driven development	Company-controlled development

Git provides greater flexibility, transparency, and accessibility compared to proprietary tools. Its open-source nature allows users to inspect, modify, and distribute the software, making it highly suitable for collaborative environments. While Perforce may be useful in controlled enterprise settings, Git better represents the principles of openness and shared development.

7. Shell Script Tasks

7.1 Script 1 — System Identity Report

Objective:

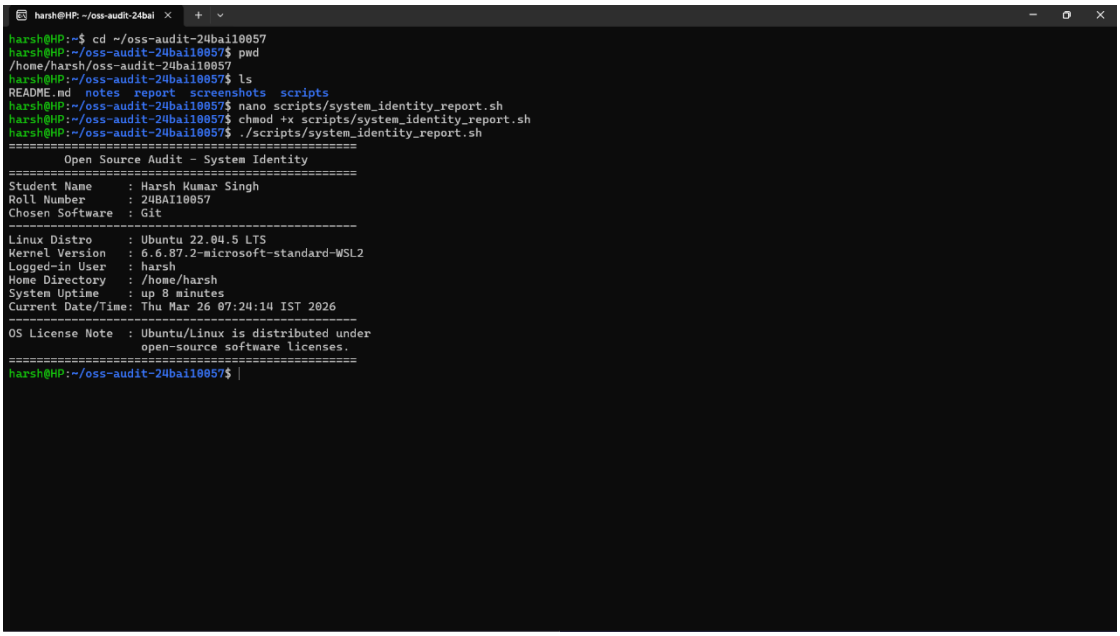
This script displays system information such as Linux distribution, kernel version, user, home directory, uptime, and date/time.

Code :

```
#!/bin/bash
# Script 1: System Identity Report
# Author: Harsh Kumar Singh
# Roll No: 24BAI10057
# Purpose: Display basic Linux system information

# Storing basic project details in variables
STUDENT_NAME="Harsh Kumar Singh"
ROLL_NO="24BAI10057"
SOFTWARE="Git"

# Collecting system information using command substitution
DISTRO_NAME=$(grep '^PRETTY_NAME=' /etc/os-release | cut -d= -f2 | tr -d '"')
KERNEL_VERSION=$(uname -r)
CURRENT_USER=$(whoami)
HOME_DIRECTORY=$HOME
UPTIME_INFO=$(uptime -p)
CURRENT_DATE_TIME=$(date)

# Displaying the report
echo "==================================================="
echo "          SYSTEM IDENTITY REPORT"
echo "==================================================="
echo "Student Name      : $STUDENT_NAME"
echo "Roll Number       : $ROLL_NO"
echo "Chosen Software   : $SOFTWARE"
echo "---------------------------------------------------"
echo "Linux Distribution: $DISTRO_NAME"
```

```
echo "Kernel Version    : $KERNEL_VERSION"
echo "Current User      : $CURRENT_USER"
echo "Home Directory    : $HOME_DIRECTORY"
echo "System Uptime     : $UPTIME_INFO"
echo "Current Date/Time : $CURRENT_DATE_TIME"
echo "--------------------------------------------------"
echo "License Note      : Linux is open-source software"
echo "                    distributed under free and"
echo "                    open-source licenses."
echo "==================================================="
```

Output:



Explanation

This script displays system information such as Linux distribution, kernel version, current user, home directory, uptime, and date/time. It uses variables and command substitution to retrieve system details and formats the output using echo.

Concepts used

Variables, echo, command substitution

7.2 Script 2 — FOSS Package Inspector

Objective:
This script checks whether Git is installed and displays package details and version information.

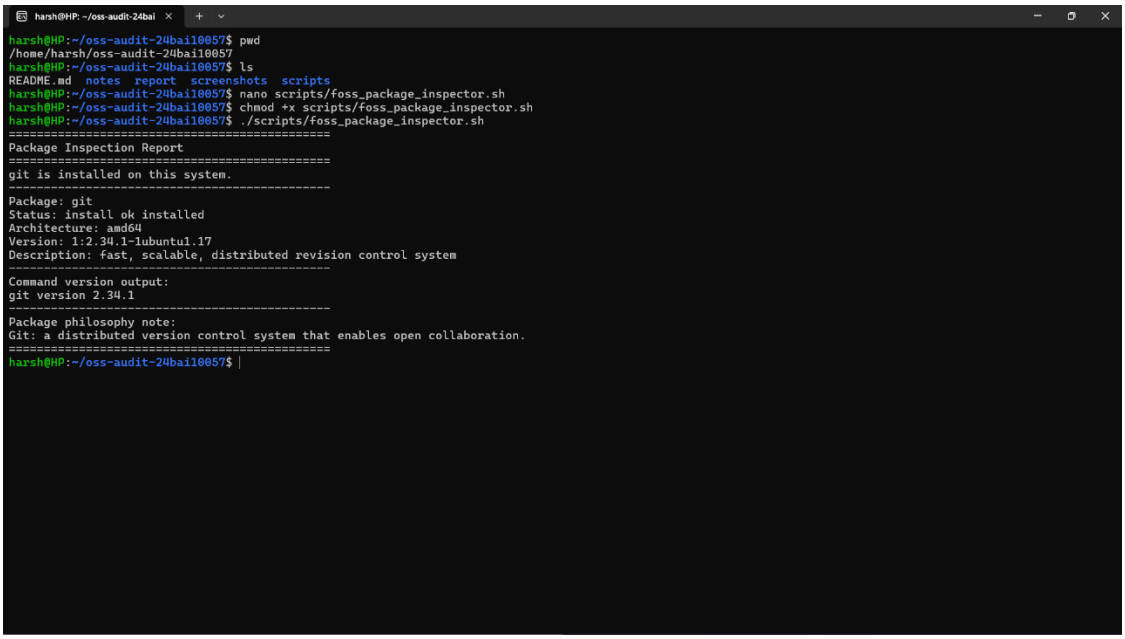
Code:

```
#!/bin/bash
```

```
# Script 2: FOSS Package Inspector
# Author: Harsh Kumar Singh
# Roll No: 24BAI10057
# Course: Open Source Software
# Purpose: Check whether a package is installed and show package details

# ----------------------------
# Chosen package
# ----------------------------
PACKAGE="git"

echo "==================================================="
echo "          FOSS PACKAGE INSPECTOR"
echo "==================================================="

# ----------------------------
# Check if package is installed
# ----------------------------
if dpkg -l | grep -qw "$PACKAGE"; then
   echo "$PACKAGE is installed on this system."
   echo "---------------------------------------------------"
   echo "Package Details:"
   dpkg -s "$PACKAGE" | grep -E 'Package:|Status:|Version:|Architecture:|Description:'
   echo "---------------------------------------------------"
   echo "Command Output:"
   git --version
else
   echo "$PACKAGE is NOT installed on this system."
fi

echo "---------------------------------------------------"
echo "Package Purpose:"

# ----------------------------
# Case statement for package meaning
# ----------------------------
case $PACKAGE in
   git)
      echo "Git is a distributed version control system used"
      echo "for tracking code changes and enabling collaboration."
      ;;
   apache2)
      echo "Apache is an open-source web server."
      ;;
   mysql-server)
```

```
        echo "MySQL is an open-source relational database system."
        ;;
    firefox)
        echo "Firefox is an open-source web browser."
        ;;
    *)
        echo "No package description available."
        ;;
esac

echo "==================================================="
```

Output:



Explanation

This script checks whether Git is installed on the system using dpkg and grep. If found, it displays package details and version information. It also uses a case statement to describe the purpose of the package.

Concepts used

If-else, case, dpkg, pipe, grep

7.3 Script 3 — Disk and Permission Auditor

Objective:
This script analyzes disk usage and displays file and directory permissions using a loop.

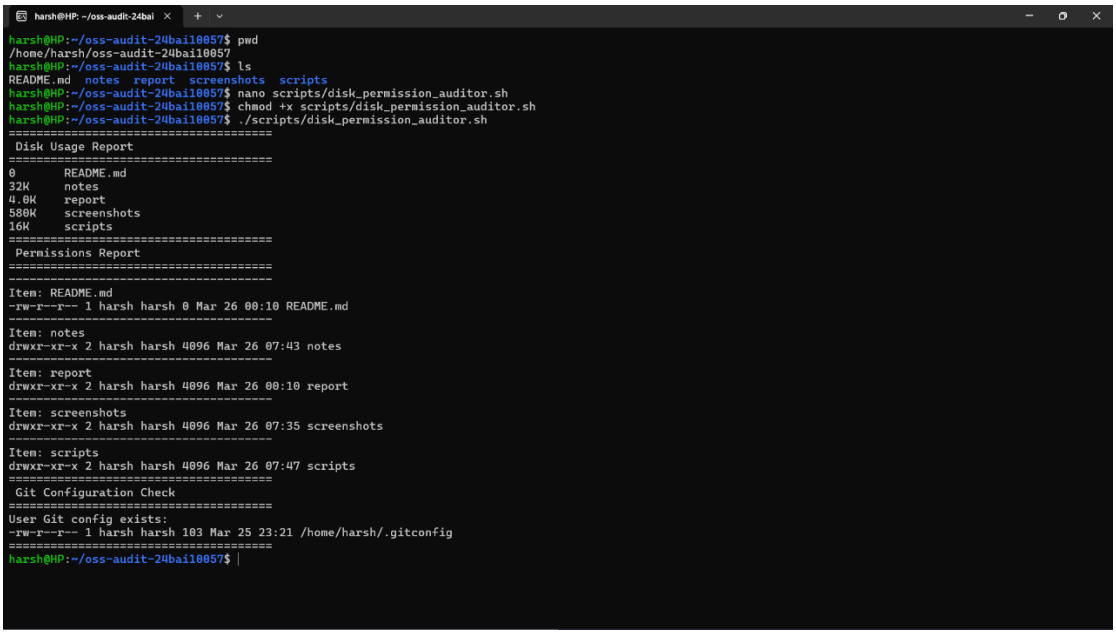
Code:

```
#!/bin/bash
```

```
# Script 3: Disk and Permission Auditor
# Author: Harsh Kumar Singh
# Roll No: 24BAI10057
# Course: Open Source Software
# Purpose: Display disk usage and permissions of files/directories

echo "==================================================="
echo "        DISK AND PERMISSION AUDITOR"
echo "==================================================="

echo "Disk Usage in Current Directory:"
echo "---------------------------------------------------"
du -sh ./* 2>/dev/null

echo "---------------------------------------------------"
echo "File and Directory Permissions:"
echo "---------------------------------------------------"

# Loop through each item in current directory
for item in *; do
   if [ -e "$item" ]; then
      echo "Item: $item"
      ls -ld "$item"
      echo "---------------------------------------------------"
   fi
done

echo "Git Configuration Check:"
echo "---------------------------------------------------"

# Check user Git config file
if [ -f ~/.gitconfig ]; then
   echo "User-level Git config found:"
   ls -l ~/.gitconfig
else
   echo "No user-level Git config found."
fi

# Check system Git config file
if [ -f /etc/gitconfig ]; then
   echo "System-level Git config found:"
   ls -l /etc/gitconfig
else
   echo "No system-level Git config found."
fi
```

echo "==========="

Output:



Explanation

This script analyzes disk usage and displays file and directory permissions. It uses a loop to iterate through items and shows detailed permission information using ls. It also checks for Git configuration files.

Concepts used

For loop, if condition, du, ls, file checks

7.4 Script 4 — Log File Analyzer

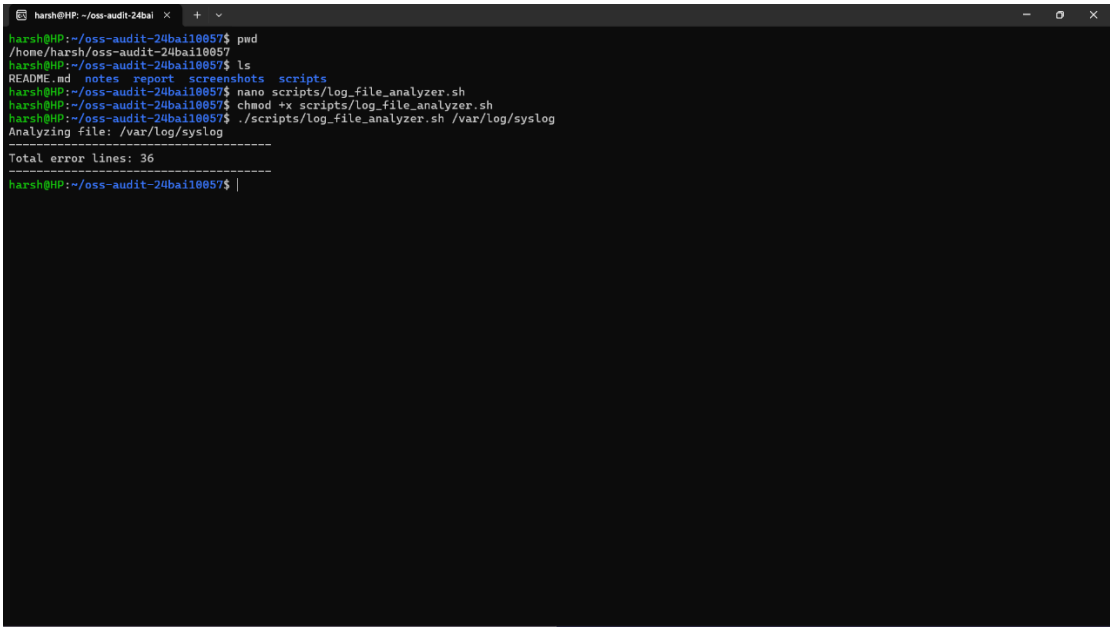
Objective:
This script reads a log file and counts the number of lines containing the word "error".

Code:

```
#!/bin/bash
# Script 4: Log File Analyzer
# Author: Harsh Kumar Singh
# Roll No: 24BAI10057
# Course: Open Source Software
# Purpose: Count lines containing the word "error" in a log file

# Check if file argument is provided
if [ -z "$1" ]; then
   echo "Usage: $0 <logfile>"
```

```
    exit 1
fi

LOG_FILE="$1"
ERROR_COUNT=0
TOTAL_LINES=0

# Check if the file exists
if [ ! -f "$LOG_FILE" ]; then
    echo "Error: File does not exist."
    exit 1
fi

echo "==================================================="
echo "             LOG FILE ANALYZER"
echo "==================================================="
echo "Analyzing file: $LOG_FILE"
echo "---------------------------------------------------"

# Read the file line by line
while read -r line; do
    TOTAL_LINES=$((TOTAL_LINES + 1))

    if [[ "$line" == *"error"* || "$line" == *"ERROR"* ]]; then
        ERROR_COUNT=$((ERROR_COUNT + 1))
    fi
done < "$LOG_FILE"

echo "Total lines in file   : $TOTAL_LINES"
echo "Lines containing error: $ERROR_COUNT"
echo "==================================================="
```

Output:



Explanation

This script reads a log file line by line and counts how many lines contain the word "error". It uses a loop and condition checks to process each line and display a summary.

Concepts used

While loop, if condition, arguments, counters

7.5 Script 5 — Manifesto Generator

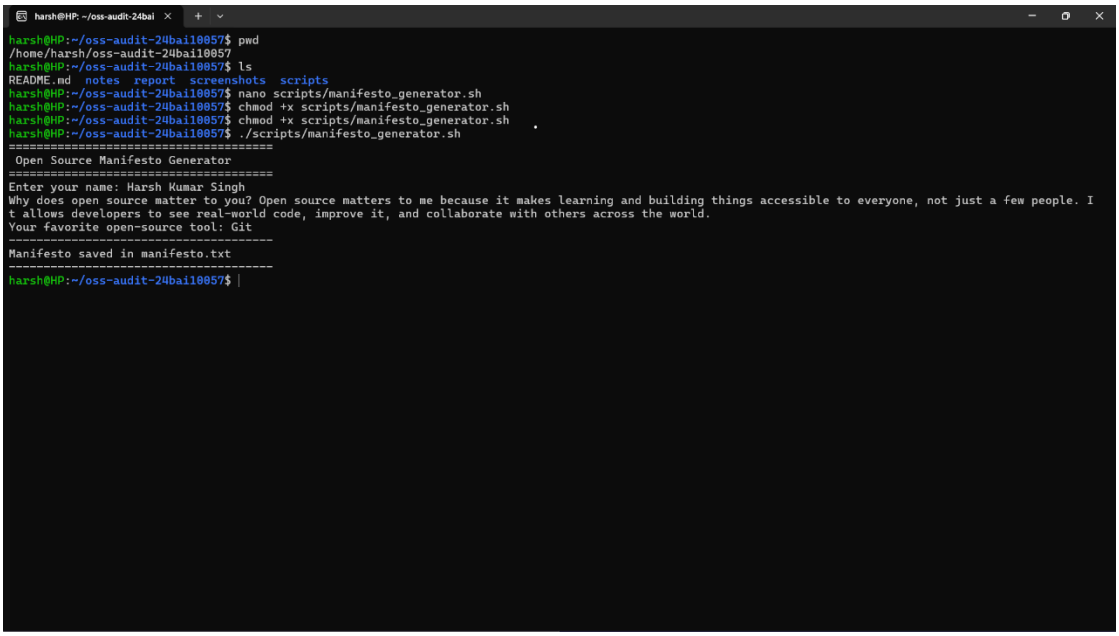
Objective:
This script takes user input and generates a short manifesto saved to a text file.

Code:

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
# Author: Harsh Kumar Singh
# Roll No: 24BAI10057
# Course: Open Source Software
# Purpose: Generate a short open-source manifesto and save it to a text file

echo "==================================================="
echo "      OPEN SOURCE MANIFESTO GENERATOR"
echo "==================================================="

# Take user input
read -p "Enter your name: " NAME
read -p "Why does open source matter to you? " REASON
read -p "What is your favorite open-source tool? " TOOL
```

```
# Output file name
OUTPUT_FILE="manifesto.txt"

# Write manifesto content to file
echo "==================================================" > "$OUTPUT_FILE"
echo "          OPEN SOURCE MANIFESTO" >> "$OUTPUT_FILE"
echo "==================================================" >> "$OUTPUT_FILE"
echo "Name          : $NAME" >> "$OUTPUT_FILE"
echo "Reason        : $REASON" >> "$OUTPUT_FILE"
echo "Favorite Tool  : $TOOL" >> "$OUTPUT_FILE"
echo "Date Generated : $(date)" >> "$OUTPUT_FILE"
echo "--------------------------------------------------" >> "$OUTPUT_FILE"
echo "I believe open source matters because it gives" >> "$OUTPUT_FILE"
echo "people the freedom to learn, improve, and share" >> "$OUTPUT_FILE"
echo "software openly. It helps students, developers," >> "$OUTPUT_FILE"
echo "and organizations collaborate in a transparent way." >> "$OUTPUT_FILE"

echo "Manifesto has been saved to $OUTPUT_FILE"
echo "==================================================="
```

Output:



Explanation

This script takes user input and generates a short manifesto saved to a text file. It uses input commands and file redirection to store the generated content along with the current date.

Concepts used

Read, variables, redirection, date

8. Conclusion

This project helped me understand open-source software beyond just coding. Git is not only a tool for version control but also a reflection of open collaboration and transparency.

Through this project, I learned how licensing affects software usage, how Linux systems manage software internally, and how open-source tools form interconnected ecosystems.

Working with shell scripts also improved my understanding of Linux commands and automation. Overall, this project provided both theoretical and practical insights into open-source software.

9. References

1. Git Official Documentation.
2. GNU Project. GNU General Public License version 2.
3. GNU Project. The Free Software Definition.
4. Ubuntu Linux manual pages and package management documentation.
5. Open Source Audit project brief and course instructions.
6. Terminal outputs collected using git, dpkg, apt, and related Linux commands.
7. Practical observations from shell script execution on Ubuntu Linux.